

花卉分类器 · 训练过程记录

本文记录四次真实训练的完整配置、逐轮数据、曲线与结论。

数据来源： docs/logs/history_Adam.json 、 docs/logs/history_SGD.json 、 docs/logs/history_AdamW_v2.json

(由 train.py / train_v2.py 自动落盘) , 以及实验 0 的终端输出 (当时落盘代码尚未加入) 。

用途：既是提交材料，也是"以后怎么调参"的证据链 —— 想重画曲线不必重训。

一、实验总览

前三次训练固定随机种子 42，因此初始权重完全相同，唯一变量是优化器与学习率。

第四次（实验 3）在此之上叠加了"训练策略 + 数据增强"，用来验证"v1 的天花板是不是训练配置造成的"。

#	配置	末轮 train acc	末轮 val acc	最佳 val acc	结论
0	SGD lr=1e-4, momentum=0.9	0.2973	0.2939	0.3186	✗ 完全没学到
1	Adam lr=1e-4	0.7687	0.6749	0.6946 (epoch 12)	✓ 提交基线
2	SGD lr=1e-2, momentum=0.9	0.7023	0.6732	0.6732 (epoch 15)	✓ 收敛, 略逊
3	AdamW + 余弦退火 + 强增强 + label smoothing, 35 轮	0.8313	0.7849	0.7865 (epoch 33)	✓ 进阶实验

提交基线：实验 1 (Adam) , test 集准确率 71.28%。

进阶实验：实验 3, test 集准确率 79.10% (+7.82 pt) , 结构一行未改，见第六节。

一句话结论：调参的优先级不是"选哪个优化器更优"，而是先确认超参数之间是否配对。

配对错误带来的差距 (约 38 个百分点) 远大于优化器之间本身的差距 (约 2 个百分点) 。

把这一条修好之后，剩下的提升空间在"训练策略"上，而不在"换结构"上。

二、统一训练配置（实验 0 / 1 / 2）

项	取值
数据集	train 2425 / val 609（每类各 20%，分层）
batch_size	32
epochs	15
损失函数	<code>nn.CrossEntropyLoss()</code>
随机种子	42
num_workers	2
计算设备	CUDA（RTX 5060 Laptop）
单轮耗时	约 70 s（总计约 17 min / 组）

训练循环五步（顺序不可换）：

```
for inputs, labels in loader:
    optimizer.zero_grad()           # ① 清梯度（backward 是累加，不是赋值）
    outputs = model(inputs)          # ② 前向
    loss = criterion(outputs, labels) # ③ 算损失
    loss.backward()                  # ④ 反向传播
    optimizer.step()                 # ⑤ 更新参数
```

两处开关：训练前 `model.train()`，评估前 `model.eval()`。

`eval()` 漏掉的后果：验证时 Dropout 仍在随机丢弃，

`if val_acc > best_val_acc` 保存下来的会是“**运气最好的一轮**”而非真正最优模型。

实验 3 改了上面这张表里的好几项，差异单独列在 6.2，不改动本节的提交基线。

三、实验 0：SGD lr=1e-4 —— 一次「模型不学习」的完整诊断

3.1 现象

```
epoch 1/15  train loss 1.6099  acc 0.1769 | val loss 1.6098  acc 0.1823
epoch 15/15 train loss 1.6010  acc 0.2973 | val loss 1.6007  acc 0.2939
```

3.2 两条关键证据

证据 A: loss 几乎没动。

15 轮 1140 个训练步，loss 只从 1.6099 降到 1.6010（降幅 0.0089）。
而 $\ln(5) = 1.6094$ 正是"五分类瞎猜"的损失值——
说明模型输出几乎仍是"五个类各 20%"的均匀分布，**参数压根没被推动过。**

证据 B: train acc $\approx$ val acc。

0.2973 vs 0.2939，两个数几乎一样。

这一步的判读方法比结果更重要：

- 若 train 高、val 低 $\rightarrow$ **过拟合**（模型记住了训练集，没学到规律）；

- 若 train 和 val 都低（本例） $\rightarrow$ **欠拟合**（模型连训练集都记不住）。

本例是欠拟合，所以**问题不在数据、不在网络结构**，只能出在"参数有没有被推动"上。

3.3 根因：学习率与优化器不配对

优化器	怎么决定每步走多远	常用 lr
Adam	按每个参数的梯度历史 自动缩放 步长，所以能吃很小的 lr	1e-4 ~ 3e-3
SGD	完全按给定的 lr 走， 没有任何自适应	1e-2 ~ 1e-1 （配 momentum）

lr=1e-4 是给 Adam 的量级；配给 SGD 相当于"开着一辆没有油门辅助的车，却只踩 1% 的油门"——差的不是 10%，是 100 倍。

3.4 修法与验证

两条路（见实验 1 / 实验 2）：

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
```

路线 A

路线 B

两条都跑通了，**验证了根因确实是 lr 而非"SGD 不行"。**

四、实验 1：Adam lr=1e-4 (提交基线)

4.1 逐轮数据

epoch	train loss	train acc	val loss	val acc
1	1.4631	0.3361	1.3233	0.3974
2	1.2574	0.4482	1.3128	0.4269
3	1.1559	0.5155	1.1157	0.5271
4	1.0609	0.5571	1.1604	0.4975
5	1.0221	0.5876	1.0329	0.6043
6	0.9291	0.6375	0.9591	0.6453
7	0.8750	0.6487	1.0417	0.5961
8	0.8561	0.6565	0.8917	0.6700
9	0.8063	0.6903	0.9793	0.6683
10	0.7608	0.7060	0.9996	0.6585
11	0.7072	0.7303	0.8855	0.6749
12	0.6830	0.7460	0.8417	0.6946 ← best
13	0.6414	0.7604	0.9619	0.6535
14	0.6242	0.7612	0.9303	0.6814
15	0.6310	0.7687	0.9394	0.6749

4.2 观察

1. **收敛正常**：train acc 从 0.34 一路爬到 0.77，没有任何"卡住"的迹象；
2. **最佳出现在 epoch 12** (val 0.6946)，此后 val 在 0.65~0.68 之间波动 →
best_model.pth 保存的正是这一轮，说明"按 val 选最优"的机制在正常工作；
3. **过拟合起点约在 epoch 8**：val_loss 从第 8 轮起在 0.88~1.00 横盘甚至回升，
而 train_loss 继续降到 0.63 → 模型开始"记"训练集了；
4. **train / val 差距逐轮扩大** (末轮 0.094) → 轻度过拟合。Dropout(0.5) 已在抑制，
但 5800 万参数配 2425 张图，这个趋势是预期的；
5. **val 到末轮仍未见顶** → EPOCHS=15 偏少，值得试 25 轮。
(这条就是后来实验 3 的起点 —— **第 5 点是对的，见第六节。**)

4.3 产物

- 最优权重: `checkpoints/best_model.pth` (233 MB, $58,301,829 \times 4$ 字节)
- 曲线图: `docs/figures/fig_curve_Adam.png`
- 训练历史: `docs/logs/history_Adam.json`

五、实验 2: SGD $lr=1e-2$, momentum=0.9

5.1 逐轮数据

epoch	train loss	train acc	val loss	val acc
1	1.5930	0.2726	1.5193	0.3563
2	1.4095	0.3876	1.3423	0.4647
3	1.2778	0.4491	1.2525	0.4647
4	1.2210	0.4565	1.1829	0.4828
5	1.1247	0.5340	1.1772	0.5090
6	1.0901	0.5625	1.0778	0.5550
7	1.0474	0.5757	1.1190	0.5484
8	1.0003	0.6148	0.9777	0.6453
9	0.9147	0.6470	1.0566	0.6059
10	0.9520	0.6272	1.0329	0.6289
11	0.8845	0.6623	0.9482	0.6585
12	0.8167	0.6920	0.9320	0.6371
13	0.7995	0.6845	0.9654	0.6371
14	0.7948	0.6866	0.9124	0.6716
15	0.7807	0.7023	0.8792	0.6732 ← best

5.2 观察

1. 把 lr 提到 $1e-2$ 后, SGD 立刻活了过来: val acc 从实验 0 的 0.29 跳到 0.67 ——
同一份数据、同一个网络、同一个种子, 只改了 lr ;
2. 上升比 Adam 慢但更稳: SGD 的 val 曲线抖动更小, 末轮还在爬 (0.6732 就是最佳);
3. 末轮 loss 仍在下降 (train 0.78 / val 0.88) → 若给更多轮次, SGD 有可能追平甚至反超 Adam;

4. 训练集 acc (0.7023) 低于 Adam (0.7687) → SGD 在小数据量上"记得更少", 这正是它泛化有时更好的原因, 但在 15 轮的预算内还没体现出来。

5.3 产物

- 曲线图: docs/figures/fig_curve_SGD.png
- 训练历史: docs/logs/history_SGD.json

⚠ 注意: checkpoints/best_model.pth 文件名是固定的, 实验 2 覆盖了实验 1 的权重。
实验 1 (Adam) 的训练历史完整保留在 json 中, 需要其权重时重跑一次 train.py 即可。
这是一个已记录的已知问题 (见提交报告 7.5), 改进方向是把 checkpoint 也按优化器命名。
—— 实验 3 已经用了这个改进: 它的权重叫 best_model_v2.pth, 不会覆盖上面任何一份。

六、实验 3：训练策略与数据增强（进阶实验）

目的: 实验 1 的第 5 条观察写着"val 到末轮仍未见顶"。那就把轮数加上去, 看看 "71.28% 是模型结构的天花板, 还是训练配置的天花板"。

结论先行: test 71.28% → 79.10% (+7.82 pt), 而网络结构一行都没改。
所以那个天花板不是模型造成的, 是"训练没跑完"造成的。

6.1 先搞清楚：什么是「增强算子」

这一小节是概念补课。"数据增强"这四个字听了很久, 但它到底在代码里长什么样、一个"算子"对一张图做了什么, 值得单独讲透 —— 因为实验 3 的一半提升都挂在它身上。

(1) 原理：三个名字说的是同一件事

他可能在不同教材里见过好几个叫法, 先列出来, 它们是同一个东西:

名字	出处
数据增强 / data augmentation	最常见的叫法
增强算子 / augmentation operator	"算子"就是"一个变换操作", 即 transforms 里的一个函数
数据扩增 / 图像增广	同一件事的另译名

一句话定义: 在图片喂进模型之前, 对训练图做随机变换, 让模型每一轮看到的"同一张图"都不一样。

为什么需要它（不这么做会怎样）：

模型有 5830 万个参数，训练集只有 2425 张图。如果每轮都喂一模一样的原图，模型最快的收敛方式是**"把 2425 张图背下来"**——训练集准确率会冲到 0.95+，但换了没见过的图就原形毕露。这叫**过拟合**。

增强让**"每一轮都是新图"**（等效于把 2425 张图变成无限流），**模型就没法背了，只能去学真正的规律。**

(2) 增强算子分三类

类别	作用在哪个维度	例子
几何类	改变像素的位置	水平翻转、旋转、随机裁剪、缩放
颜色类	改变像素的值	亮度 / 对比度 / 饱和度 / 色调抖动
遮挡类	抹掉一块	RandomErasing、Cutout

(3) Python 语法落地块

```
from torchvision import transforms

tf = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.25),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
    transforms.RandomErasing(p=0.25),
])
```

Compose = 把算子串成一条流水线

几何：随机裁一块再放大回 224

几何：一半概率左右翻转

颜色：亮度抖动 ±25%

← 分界线：PIL 图 → 张量

张量阶段

遮挡：必须在 ToTensor 之后！

● 最容易踩的一条：算子分成两个阶段 ——

阶段	吃进去的	能放哪些算子
ToTensor() 之前	PIL 图像 (Image.open 的产物)	翻转 / 旋转 / 裁剪 / 颜色抖动 / RandAugment
ToTensor() 之后	张量 (形状 (3,224,224))	Normalize、RandomErasing

RandomErasing 放错位置会直接报错 —— 因为它要按张量下标去"擦"，PIL 图没有那种下标。

形状进出： Image.open(...) → (500, 313) 的 PIL 图 → ToTensor() → (3, 224, 224) → 加 batch 维 → (32, 3, 224, 224) 。

(4) 怎么量化"这个算子到底改了什么" —— 逐像素差

"看着不一样"不算证据。用**两次跑同一张图**的**平均像素绝对差**来量（越小越像）：

算子（单用）	两次差异	是否随机
什么都不做（只 Resize+ToTensor）	0.00000	否
Resize+ToTensor+Normalize（= 评估管道）	0.00000	否（每次完全一致）
RandomHorizontalFlip(p=0.5)	0.26654	是
RandomRotation(15)	0.04303	是
ColorJitter(0.25, 0.25, 0.25)	0.20352	是
RandomResizedCrop(224, scale=(0.7, 1.0))	0.14439	是
RandAugment(num_ops=2, magnitude=7)	0.30851	是
RandomErasing(p=0.25, scale=(0.02, 0.15))	0.00000	是（这次两次都没触发）

最后一行是个重要的教学点：`p=0.25` 是概率型算子，单跑两次可能都没生效，于是差异为 0。所以"差异为 0"不等于"这个算子没用"——要跑很多次才看得出。
→ 判据是"跑 20 次有没有出现过不同"，不是"跑 2 次差多少"。

同一张图被增强成多少种模样（各跑 20 次，两两组合共 190 对）：

管道	两两差异均值	最小	最大	完全相同的组合
v1 train_transform（3 个算子）	1.00067	0.06756	1.44598	0 / 190
v2 train_transform_v2（6 个算子）	1.21358	0.18734	2.10733	0 / 190

190 对组合里一对相同的都没有 → 增强真的在生效，而且 v2 的"变化幅度"明显更大。

(5) 挂回本项目

文件	内容
src/data/transforms.py	v1 的 3 个算子： RandomHorizontalFlip + RandomRotation(15) + ColorJitter
src/data/transforms_v2.py	v2 新增 3 个： RandomResizedCrop + RandAugment + RandomErasing

铁律（两版都适用）：随机算子只能进 `train_transform`。
`eval_transform` 一点随机性都不能沾 —— 上表第 2 行就是证据，评估管道两次跑的差异必须是 0.00000，否则验证集/测试集的成绩每次都不同，根本不可比。

6.2 配置差异 (v1 vs v2)

项	v1 (实验 1/2)	v2 (实验 3)	属于哪档
epochs	15	35	A
优化器	Adam(lr=1e-4)	AdamW(lr=1e-4, weight_decay=1e-4)	C
学习率调度	无 (恒定 1e-4)	CosineAnnealingLR	A
损失函数	CrossEntropyLoss()	CrossEntropyLoss(label_smoothing=0.1)	C
训练增强	翻转 + 旋转 + ColorJitter	再加 RandomResizedCrop + RandAugment + RandomErasing	B
评估增强	Resize + Normalize	同左 (一个字没动)	—
权重文件	best_model.pth	best_model_v2.pth (不覆盖)	—
单轮耗时	约 70 s	约 90 s (增强变重了)	—

(A / B / C 三档的含义见 6.4 起的归因部分。)

6.3 逐轮数据 (35 轮)

epoch	train loss	train acc	val loss	val acc
1	1.5396	0.2981	1.4444	0.3760
2	1.3944	0.4157	1.3232	0.4351
3	1.3376	0.4351	1.2980	0.4926
4	1.2904	0.4767	1.2882	0.4844
5	1.2475	0.5361	1.1793	0.5813
6	1.1995	0.5637	1.1755	0.5993
7	1.1705	0.5773	1.1049	0.6338
8	1.1249	0.6173	1.0950	0.6141
9	1.1049	0.6198	1.1102	0.6486
10	1.0887	0.6334	1.0483	0.6404
11	1.0599	0.6528	1.1099	0.6420
12	1.0439	0.6647	1.0281	0.7028
13	1.0285	0.6619	1.0774	0.6486
14	1.0025	0.6792	0.9805	0.7094
15	0.9903	0.6920	1.0146	0.6995
16	0.9633	0.7010	0.9635	0.7422
17	0.9653	0.7010	0.9689	0.6913
18	0.9281	0.7233	0.9793	0.6979
19	0.8970	0.7522	0.9630	0.7143
20	0.9070	0.7443	0.9257	0.7471
21	0.8834	0.7443	0.9077	0.7521
22	0.8577	0.7658	0.9003	0.7406
23	0.8286	0.7794	0.9309	0.7258
24	0.8365	0.7761	0.8778	0.7586
25	0.8164	0.7893	0.8795	0.7570
26	0.8059	0.7880	0.8495	0.7767
27	0.7821	0.8078	0.8591	0.7783

epoch	train loss	train acc	val loss	val acc
28	0.7578	0.8177	0.8657	0.7816
29	0.7668	0.8214	0.8562	0.7767
30	0.7566	0.8239	0.8476	0.7783
31	0.7502	0.8264	0.8498	0.7767
32	0.7478	0.8280	0.8480	0.7849
33	0.7502	0.8202	0.8429	0.7865 ← best
34	0.7478	0.8384	0.8429	0.7849
35	0.7357	0.8313	0.8428	0.7849

6.4 归因之一：9.2 个点是从哪几轮挣来的

这是最关键的发现。把 v1 和 v2 的验证曲线按 epoch 对齐：

epoch	v1 val acc	v2 val acc	差距
12	0.6946 (v1 最佳)	0.7028	+0.8 pt
15	0.6749 (v1 结束)	0.6995	+2.5 pt
20	— (v1 没跑到)	0.7471	—
25	—	0.7570	—
30	—	0.7783	—
33	—	0.7865 (v2 最佳)	—

读出来的事实：

- v2 跑到第 12 轮，才刚追平 v1 的最终成绩；到第 15 轮（v1 的终点）也只领先 2.5 个点。
- 换句话说：如果 v1 多训 15 轮，它大概也能到 0.70 左右——但仅此而已。
- 多出来的 9.2 个点，全部发生在 v1 从未走到过的第 16~35 轮。

这个事实直接决定了归因的方向：主力是"让训练继续下去"的那两项，而不是增强。

下面按 A / B / C 三档拆开。

6.5 A 档（主力）：35 轮 + 余弦退火

这两项有直接证据，是涨点的主力。

① 轮数 15 → 35

v1 跑到第 15 轮时，`train_loss` 还在从 0.63 往下走、`val_acc` 也还在爬（见 4.2 第 5 条）——
曲线根本没到平台期，是被 `range(EPOCHS)` 硬掐断的。这是典型的欠训练，加轮数直接对症。

而 v2 跑到第 33 轮之后，曲线明显走平（0.7865 → 0.7849 → 0.7849），
说明**35 轮已经把这条曲线跑完了**，再加轮数收益很小。两次对比正好确认了这一点。

② 余弦退火 `CosineAnnealingLR`（最被低估的一项）

原理一句话：v1 的学习率从头到尾钉死在 `1e-4`。训练后期模型已经在最优点附近了，
但每次更新还是迈那么大的步子——**一脚跨过去，再一脚跨回来。**
这就是 v1 曲线上那些锯齿的来源。

让 lr 随时间走一条**余弦曲线**，从 `1e-4` 平滑降到 ~0：

$$lr(t) = lr_min + 0.5 \times (lr_max - lr_min) \times (1 + \cos(\pi \times t / T))$$

- `t=0` 时 `cos(0)=1` → `lr = lr_max`（起点步长最大）
- `t=T` 时 `cos(π)=-1` → `lr = lr_min`（终点步长最小）
- 中间的下降是"两头缓、中间陡"的余弦形状

Python 落地（就是 v2 里加的那几行）：

```
from torch.optim.lr_scheduler import CosineAnnealingLR

scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS_V2)  # T_max = 总轮数

for epoch in range(1, EPOCHS_V2 + 1):
    train_one_epoch(...)      # lr 从 1e-4 开始
    evaluate(...)
    scheduler.step()          # ← 每个 epoch 走一步，lr 沿余弦降到约 0
```

⚠️ 两个容易错的点：

- `scheduler.step()` 放在 **epoch 循环里**（每轮一次），不是放在 `batch` 循环里；
- `T_max` 要等于**总轮数**（这里是 35），这样 lr 恰好在最后一轮降到最低。

数据证据（后期震荡幅度）：

	后期 val acc 波动范围	幅度
v1 (lr 恒定)	0.6535 ↔ 0.6946	4.1 %
v2 (余弦退火)	0.7767 ↔ 0.7865	1.0 %

震荡收窄到约四分之一。步子放慢了，模型才能停在最优点上。

6.6 B 档：三个强增强算子

B 档扮演的是“守门员”，不是“前锋”。它不直接涨分，但没有它，A 档会反噬。

三个算子各自在做什么

① `RandomResizedCrop(224, scale=(0.7, 1.0))` —— 替掉 v1 的固定 `Resize`

- v1 的做法是 `Resize((224,224))`：把整张图压缩成 224。
后果是**每张图的构图永远一样**——花永远在画面正中、永远占同样的比例。
模型学到的可能是**“正中间那团像素是什么颜色”**，而不是**“这朵花长什么样”**。
- v2 换成随机裁一块（至少保留 70% 面积）再放大回 224：
模型会见到花偏在角落、见到只拍到半朵花、见到远近不同的构图。
- 这是小数据集上**性价比最高的单点增强**（分类任务里几乎是默认操作）。

② `RandAugment(num_ops=2, magnitude=7)` —— 一行顶十几个算子

- 它内部有一个**算子池**（旋转、剪切、色调分离、日光化、对比度、锐度.....十几个），
每次**随机挑 2 个**执行，强度由 `magnitude` 控制（范围 0~9，这里取 7，属中等偏强）。
- 代码只有一行，但等价于手工堆十几个算子的组合，而且**组合还是随机的**。
- ⚠️ **强度不能往上加到 9**：池子里有 `solarize` / `posterize` 这类
会破坏颜色信息的算子，调太猛会让“黄花”和“橙花”变得难以区分——
对这个“按颜色区分花种”的任务，这是负优化。

③ `RandomErasing(p=0.25, scale=(0.02, 0.15))` —— 随机涂掉一小块

- 每 4 张图擦 1 张，擦掉的面积占整图的 2%~15%。
- 作用：逼模型**别只盯某个局部特征**。
比如模型可能学会了“看到中间那抹黄就是向日葵”，擦掉之后它必须去找别的证据。
- 必须在 `ToTensor()` 之后（见 6.1 的阶段表）。

为什么说 B 档是“守门员”而不是“前锋”

看这组对照 —— `train / val` 的差距：

	训练轮数	末轮 train acc	末轮 val acc	gap
v1	15	0.7687	0.6749	0.0938
v2	35	0.8313	0.7849	0.0464

训了 2.3 倍的时间，`train` 与 `val` 的差距反而缩小了一半。

这就是增强在起作用的证据：如果不加这三个算子，5830 万参数在 2425 张图上跑 35 轮，`train_acc` 早就冲到 0.95+，而 val 会往下掉 —— 那样第 16~35 轮就不是涨分，而是掉分。增强把"训得久"的副作用压住了，A 档的提升才得以兑现。

一个反直觉的副作用：v2 的开局更慢

第 1 轮	train acc	val acc
v1	0.3361	0.3974
v2	0.2981	0.3760

v2 第一轮**明显更低**。原因很简单：**每张训练图都变难了**，模型在"更难为题"上当然起步慢。**这是正常现象，不是 bug** —— 后程会反超。（第 12 轮 v2 是 0.7028，已经超过 v1 的最佳 0.6946。）

→ **结论：B 档是"必要前提"，但不是涨点的直接原因。** 它属于"间接证据"级。

6.7 C 档：`label_smoothing=0.1` 与 AdamW

这两项**没有做对照实验**，所以下面只能讲"机制上为什么有益"，**不能声称它们各贡献了几个点**。要量化它们必须做**消融实验**（ablation）：每次只关掉一项重跑、对比差值。

① `label_smoothing=0.1` —— 别让模型把话说得太满

普通交叉熵在干什么：它把正确答案当成 one-hot —— 正确类目标 1.0，其余类 0.0。于是模型只要把正确类的 logit 推得足够高，损失就会趋近于 0。

问题在哪：看这组实测数字（5 分类，模型很自信地认为是第 0 类，且答对了）：

正确类 logit	普通交叉熵 loss	<code>label_smoothing=0.1</code>
2.0	0.432653	0.592653
5.0	0.026595	0.426595
10.0	0.000182	0.800182
20.0	0.000000	1.600000

左边那一列在"又对又自信"时几乎归零 —— 也就是说，模型把 logit 从 10 推到 20，普通交叉熵**几乎不给任何惩罚**。于是训练会**无休止地推高预测值**，最终的模型对每个输入都极其"嘴硬"，泛化反而差。

`label_smoothing` 的做法是**把目标分布从 one-hot 改成带一点平摊**：

目标类别	one-hot	label_smoothing=0.1
正确类	1.0000	0.9200
其余每类 (共 4 类)	0.0000	0.0200

⚠ 注意这个分布的数字：正确类是 $(1 - \epsilon) + \epsilon/K$ ，其余是 ϵ/K —— 分母是 K (类别总数 5)，不是 K-1。这是 PyTorch 的实际实现。

效果：上表右边那一列，logit 推到 20 时损失仍有 1.6 —— **永远有个地板**。
模型没法通过"推极端"来刷损失，就不得不去学更有区分度的特征。
常用取值 0.05~0.1，代价是"训练损失的最低值会比原来高"（这不是变差了，是尺子变了）。

② AdamW(weight_decay=1e-4) —— 把"权重衰减"从梯度里拿出来

先说 weight_decay 是什么：在每次更新时，额外让权重朝 0 收缩一点点，防止某些权重变得特别大。这是一种**正则化**——抑制过拟合的手段。

Adam 与 AdamW 的差别在哪（这是 AdamW 存在的全部理由）：

Adam 会把**整个梯度（包括 weight_decay 加进去的那一项）**一起除以自适应缩放系数（即 $\sqrt{v}$ ）。而 AdamW 把 weight_decay **单独拿出来**、直接用 $lr \times weight_decay$ 衰减，不参与自适应缩放。

实测证据（把梯度设成 0，这样看到的位移只可能来自 weight_decay）：

优化器 (lr=0.1, weight_decay=0.1)	起点	5 步后	位移
Adam (无衰减)	1.0000	1.000000	0.00 %
Adam(weight_decay=0.1)	1.0000	0.507964	49.20 %
AdamW(weight_decay=0.1)	1.0000	0.950990	4.90 %

同样是 weight_decay=0.1，Adam 的实际衰减强度是 AdamW 的 10 倍 —— 因为 Adam 的自适应缩放把这一项也放大了。
所以"Adam + weight_decay"这个组合的衰减强度是**失控的**，而 AdamW 才是"按你写的数字衰减"。

落到本项目：这里 weight_decay=1e-4 很小，属于"轻微正则"，和 label_smoothing 一起作为 C 档的锦上添花。

6.8 最终成绩

指标	v1 (Adam, 15 轮)	v2 (35 轮 + 强增强 + 余弦退火)	变化
test 准确率	0.7128	0.7910	+7.82 pt
test loss	0.7576	0.5797	-0.178
val 最佳	0.6946	0.7865 (epoch 33)	+9.19 pt
末轮 train acc	0.7687	0.8313	+6.26 pt
train / val gap	0.0938	0.0464	减半

曲线对比图：`docs/figures/fig_compare_v1_v2.png`、`docs/figures/fig_curve_AdamW_v2.png`。

关键观察：v1 那条曲线在第 12-15 轮已经开始**震荡下坠**（学习率不衰减的典型症状），而 v2 一路平滑爬升到第 33 轮才收平。loss 更明显——v1 在 0.84~1.00 之间反复横跳，v2 单调降到 0.84。

一个诚实的负结果：TTA 在这里无效

TTA (Test Time Augmentation) 的做法是：测试时把原图 + 水平翻转**各推理一次**，再把两个预测取平均。

方式	test 准确率
普通推理	0.7910
TTA (原图 + 水平翻转平均)	0.7859

TTA 反而低了 0.5 个百分点。 换算成绝对值就是 780 张里差了 4 张，属于**噪声范围**。

结论：**这个小数据集上 TTA 没有收益，不用。**

这条负结果也记下来——免得以后重复踩。

6.9 产物

- 最优权重：`checkpoints/best_model_v2.pth`
- 曲线图：`docs/figures/fig_curve_AdamW_v2.png`、`docs/figures/fig_compare_v1_v2.png`
- 训练历史：`docs/logs/history_AdamW_v2.json`
- 代码：`src/train_v2.py`、`src/data/transforms_v2.py`
- 重画对比图：`docs/labs/compare_v1_v2.py`

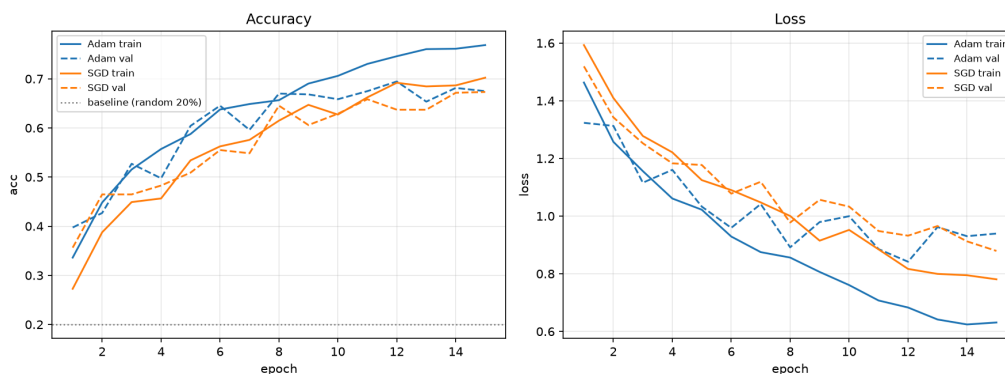
✓ v1 完全未被改动：

`train.py`、`config.py`、`data/transforms.py`、`checkpoints/best_model.pth`、原来的两张曲线图，以及 `model.py` 里那 6 处 `# TODO:` 注释，

一个字都没动。实验 3 全部走新文件、新产物名。

七、四组对照的完整结论

7.1 对照曲线



左：准确率（实线 train / 虚线 val，灰点线为 20% 随机基线）；右：损失。

该图由 `docs/labs/compare_history.py` 从 json 直接重画，不需要重训。

v1 与 v2 的对比见 `figures/fig_compare_v1_v2.png`。

7.2 结论

问题	答案
哪个优化器更好？	15 轮内 Adam 略优 (0.6946 vs 0.6732)，差距约 2 个百分点
最大的一次提升来自哪里？	不是换优化器，而是把 lr 配对 (SGD 从 0.29 → 0.67，涨了 38 个百分点)
为什么实验 0 完全学不动？	SGD + lr=1e-4 差 100 倍，每一步的更新量小到几乎为零
调参应该先做什么？	先检查超参数之间的"适配关系" (优化器 ↔ 学习率)，再比较谁更优
第二个提升来自哪里？	训练策略 (轮数 + 学习率调度)，71.28% → 79.10%， 结构一行没改
为什么结构不变也能涨 8 个点？	因为原来的 71.28% 不是结构的天花板，是 训练没跑完 的天花板

7.3 三次改动按"性价比"排序（本次学到的）

排序	改动	代价	效果	性质
1	学习率与优化器 配对	改 1 行	+38 pt	修 bug 级
2	轮数 15 → 35	改 1 个数	+7.8 pt（主力）	补足欠训练
3	余弦退火	加 3 行	附带在上一项里（震荡 4.1% → 1.0%）	提升稳定性
4	三个强增强算子	加 3 行	不直接涨分 ，是让第 2 项不反噬的前提	守门员
5	label smoothing / AdamW	各改 1 个参数	机制上有利， 未做消融，无法量化	锦上添花

7.4 后续可做的实验（尚未做）

- 1. **消融实验**：关掉 `label_smoothing` + 把 `AdamW` 退回 `Adam`，其余不变 → 量化 C 档的真实贡献；
- 2. **把 val 并回训练集**：定稿时用全部 3034 张训练（+25% 数据），val 只用于监控；
- 3. **BatchNorm**：每个卷积后插 `nn.BatchNorm2d`（AlexNet 原版没有 BN，这是它之后最重要的改进，允许更大 lr、收敛更快更稳）—— 但**偏离结构图**，需在报告中说明；
- 4. **迁移学习**：`resnet18(weights="IMAGENET1K_V1")` 换掉 `fc` 层微调，5 类花卉通常能到 92~96%。
这是工业界标准做法，但**偏离"按结构图搭 AlexNet"的题目要求**，只能作为进阶对比，不能顶替主提交。

八、训练相关的踩坑记录

8.1 验证集准确率偏低 ≠ 验证集的问题

误判："为什么验证集准确率这么低？"
实情：train acc 0.2973 ≈ val acc 0.2939 —— **两边都低**，是欠拟合，不是验证集被"区别对待"。
判读口诀：先看 train 和 val 差多少，再判断是过拟合还是欠拟合。

8.2 曲线图右边空白 —— 「陈图」陷阱

现象：训练完成后生成的曲线图，Accuracy 子图有条线，**Loss 子图整块空白**。
代码复查并模拟运行正确（两个子图各 2 条线）。

根因（时间戳钉死）：

evaluate.py 保存时间	22:40	
训练进程启动时间	~22:37	← 早了 3 分钟
曲线图生成时间	22:55	

Python 启动时只 import 一次模块 —— 进程跑起来之后再改文件，正在运行的进程毫不知情，用的还是 import 那一刻的旧版本。

规律：改完代码必须重启进程才生效。

加固： `train.py` 现在把训练历史落盘为 json，并提供 `compare_history.py` 从 json 重画 —— 以后重画曲线不必重训，也就不会再撞这个坑。

8.3 训练耗时异常的两轮

epoch 11 用了 684.9s、epoch 14 用了 1993.2s（其余约 70s）。原因是那段时间电脑休眠/挂起，而 `time.time()` 把挂起时间也算进了"这一轮耗时"。
不影响训练结果，只是计时被污染。

实验 3 里同一类坑又出现了一次： epoch 23/24 各花了 2093.8s 和 1923.5s，把总时长从预计的 53 分钟拖成 1 小时 57 分。原因这次不是休眠 —— 是**我在诊断数据管道慢时跑的几个测试脚本抢了 CPU/内存**，把正在训练的任务挤住了。

教训：跑性能测试前先确认没有训练在跑。

8.4 解释器选错

截图里出现 `C:\Users\Henry\AppData\Roaming\uv\python\...\python.exe` —— **不是项目的虚拟环境**，那里没装 torch，会报 `ModuleNotFoundError: No module named 'torch'`，极易被误判成"环境坏了"。

规矩：本项目**永远用全路径** `D:\ML\.venv_gpu\Scripts\python.exe`，不要裸敲 `python`。

8.5 「训练慢」的真凶：Windows 上的多进程 DataLoader（实验 3 新发现）

现象：训练刚起来时 GPU 利用率长期接近 0、显存只占 1.5G/8G —— GPU 全程在等数据。

第一反应（错的）：以为是数据增强太重，CPU 做不过来。

量化之后（对的）：写了个只对比 DataLoader 配置的脚本，测"读满 320 张图要多久"：

DataLoader 配置	读 320 张图
<code>num_workers=0</code> （主进程直接读）	1.8 秒 ← 最快
<code>num_workers=2</code> （v1 的配置）	约 10 秒
<code>num_workers=6</code>	31.5 秒（慢 17 倍）
<code>num_workers=12</code> + <code>persistent_workers=True</code>	直接挂死 （30 分钟打不出一个 epoch）

原因：Windows 的多进程走 **spawn 不是 fork** ——
每个 worker 都要**重新 import 一遍 torch**、每个 epoch 还要重建、
还要跨进程 pickle 搬运大张量。而这台机器是 SSD + 强单核，
主进程串行读图 183 张/秒本来就喂得饱 GPU。

顺带纠正另一个直觉错误："增强拖慢训练"是错的 ——
单张图 v1 管道 2.51 ms、v2 全管道 3.67 ms，只贵 46%，2425 张也就 9 秒。
真正的锅在 DataLoader。

结论： `num_workers=0` 。这两个诊断脚本留在
`docs/labs/bench_transforms.py` 和 `docs/labs/bench_loader.py` ，
以后遇到"训练慢"可以直接跑它们定位。

 **这一项只影响速度，对准确率没有任何影响** —— 别把它记成提分手段。

8.6 报错行 ≠ 案发行 (Python 侧，本家族第 4 次)

报错	报错指在	真凶在
<code>IndentationError: unexpected indent</code>	第 75 行 (受害者)	第 50 行 (掉回 0 列)
计数只有 tulip 有数	—	切分/复制掉出循环体
<code>AttributeError: 'bool' object has no attribute ...</code>	第 60 行 (使用处)	第 48 行 (覆盖赋值)
<code>Perhaps you forgot a comma?</code>	第 34 行 (下一个元素)	上一行漏逗号

统一收口句：「报错指的那一行，通常不是凶手那一行 —— 凶手在上游或在下游。」

九、复现方式

```
cd /d D:\ML.2

# ① 数据管道（若 processed/ 已存在可跳过）
D:\ML\.venv_gpu\Scripts\python.exe src\data\clean.py
D:\ML\.venv_gpu\Scripts\python.exe src\data\split.py
D:\ML\.venv_gpu\Scripts\python.exe src\data\dataset.py      # 自检

# ② 提交基线训练（把 src\train.py 里的 optimizer 换成目标配置）
D:\ML\.venv_gpu\Scripts\python.exe src\train.py

# ③ 评估 test 集
D:\ML\.venv_gpu\Scripts\python.exe src\evaluate.py

# ④ 重画多组对照曲线（读 docs/logs/*.json，不必重训）
D:\ML\.venv_gpu\Scripts\python.exe docs\labs\compare_history.py

# ⑤ 进阶实验（35 轮 + 强增强 + 余弦退火 + label smoothing + AdamW）
D:\ML\.venv_gpu\Scripts\python.exe src\data\transforms_v2.py    # 先验收增强管道
D:\ML\.venv_gpu\Scripts\python.exe src\train_v2.py              # 约 53 分钟
D:\ML\.venv_gpu\Scripts\python.exe docs\labs\compare_v1_v2.py  # 生成 v1/v2 对比图
```

每组训练会自动产生：

- docs/logs/history_<优化器名>.json —— 逐轮 loss / acc
- docs/figures/fig_curve_<优化器名>.png —— 该组曲线
- checkpoints/best_model.pth —— 最优权重

⚠ best_model.pth 文件名固定，**后一组会覆盖前一组**（实验 2 覆盖了实验 1）。

实验 3 用的是独立文件名 best_model_v2.pth，不受影响。

这是已记录的已知问题（见提交报告 7.5），改进方向是把 checkpoint 也按配置命名。